

Compressed Knot Groups and Degree-Budgeted $SU(2)$ Recognition

Checkpoint tradeoffs, optimal Wirtinger propagation,
and an exact two-meridian backend

Research continuation prepared with ChatGPT
for Vladimir Reshetnikov's ProveIt project

8 October 2026

Abstract

We develop a complete algebraic continuation of a certified knot-group simplifier, rather than another one-sided finite-quotient filter. For a presentation with r generators and a straight-line-program word grammar, a set of c polynomial checkpoints gives an exact nonabelian $SU(2)$ feasibility problem with $4(r + c)$ real variables and degree controlled by an explicitly computable quantity Δ . Standard existential-real algorithms then give bit complexity $\text{poly}(N)(\Delta + 2)^{O(r+c)}$, where N is the encoded presentation size. We prove that mixed checkpointing can improve the exponent asymptotically over both full substitution and full lifting, and give an optimal checkpoint dynamic program for formula trees.

For two *certified meridian* generators, we avoid real algebraic elimination altogether. Every traceless word has a normal form consisting of an integer and two parity bits. Exact feasibility reduces to a greatest-common-divisor calculation and parity checks, in time polynomial in the compressed input size even when the represented words are exponentially long. We also prove optimal fixed-seed Wirtinger degree propagation, a polynomial recognition bound for diagrams admitting two-seed propagation, and a quasi-polynomial bound for logarithmically many visible overpasses. The package supplies tested arithmetic, compilers, replayable certificates, independent symbolic comparisons, and reproducible measurements. It does not prove a general quasi-polynomial bound for the maintained recognizer or implement a general singly exponential real-feasibility engine.

Status. The results below have ordinary mathematical proofs and computational regression checks, not proof-assistant formalizations. The generic real-algebraic bound invokes established algorithms. The two-meridian arithmetic is fully implemented. Timing results concern presentation-level queries, not the repository's whole recognition pipeline.

Contents

1	Purpose, contribution, and source audit	2
1.1	The remaining interface in the maintained recognizer	2
1.2	What is established already, and what is new in this continuation	2
1.3	Current literature and the limits of this audit	2
2	Topological input and representation semantics	3
2.1	The two external topological facts	3
2.2	Unit quaternions	3
2.3	Two counterexamples that determine the API	4
3	A compressed presentation model	4
3.1	Polynomial checkpoints	4
4	The complete degree-budgeted feasibility reduction	5
4.1	Construction and correctness	5
4.2	Coefficient size and dense conversion	6
4.3	Bit complexity and the quasi-polynomial regimes	6
5	Why intermediate checkpointing is mathematically useful	7
5.1	A strict exponent separation	7
5.2	An optimal formula-tree algorithm	7
6	Optimal fixed-seed Wirtinger propagation	8
6.1	Rules and exact degree labels	8
6.2	A checkable optimality certificate	9
6.3	Visible overpasses give a concrete quasi-polynomial class	9
7	Two meridians: from real algebra to integer arithmetic	10
7.1	A complete gauge for the nonabelian traceless slice	10
7.2	The three-entry normal form	10
7.3	The complete phase test	11
7.4	Compressed bit complexity	11
8	A polynomial diagram-level certificate class	12
9	An independent univariate formulation	13
9.1	Rational parameterization	13
9.2	Gcd and Sturm correctness	13
9.3	Examples and exact cross-checks	14
10	Proof-carrying integration and failure semantics	14
10.1	The trust boundary	14
10.2	Replayable algebraic evidence	14
10.3	Budget accounting	15
11	Implementation, validation, and measured performance	15
11.1	Delivered implementation	15
11.2	Completed validation	15
11.3	Paired exact presentation queries	16
11.4	Capacity with exponentially long represented words	17
12	What this changes about the quasi-polynomial program	17
13	Further research questions and proposed work	18
14	Conclusion	19
A	Reproduction and artifact map	19
B	Deriving the torus presentation used in capacity tests	20
C	A compact independent-checking specification	20

1 Purpose, contribution, and source audit

1.1 The remaining interface in the maintained recognizer

The inspected ProveIt implementation has an exact exponential fallback and several optional, certified accelerations. Its compressed-group stage preserves a presentation while performing independently replayed transformations. Successful reduction to a cyclic knot group certifies an unknot; a stalled search is not a nontriviality certificate. The adaptive implementation changes word representation without restarting the proof trace or refilling the search allowance [1, 2]. This is an appropriate place for a *complete* continuation: after a useful presentation has been found, decide whether it has a nonabelian $SU(2)$ representation.

The distinction between *finding* and *checking* a presentation is essential. We charge its discovery and replay separately. A small presentation obtained only after an uncontrolled search is not a fast recognition algorithm. Likewise, a small grammar is not automatically a small expanded polynomial system. Repeated squaring can keep the grammar small while doubling algebraic degree at every step.

The contributions developed here are the following connected pieces. First, we specify an exact rank-checkpoint-degree cost parameter and prove the associated bit bound, including coefficient growth. Second, we turn the two-meridian case into compressed integer arithmetic with explicit positive and negative answers. Third, we provide a certified way to find and optimize expressions in fixed Wirtinger seeds and derive diagram-level restricted complexity bounds. Finally, we implement these operations without promoting an unchecked group presentation, a time limit, or a numerical failure into a knot verdict.

1.2 What is established already, and what is new in this continuation

The reduction of unknot recognition to nonabelian $SU(2)$ feasibility is not new. Meesum and Prathamesh give such an algorithm and its exponential bound [3]. We use the established representation theorems of Kronheimer–Mrowka and, for the traceless specialization, the knot case of the theorem stated by Xie–Zhang [5, 6]. Binary-dihedral representation theory is classical; the broader setting of $SU(2)$ -simple knots is discussed by Zentner [7].

Our contribution is not a claim of priority for those facts. It is an explicit compressed-input implementation and analysis, coupled to the repository’s actual presentation interface. The checkpoint exponent, its strict hybrid separation, the fixed-seed optimal-degree certificate, and the integration of exact two-meridian arithmetic are proved here. These are useful research results even though the unrestricted compression problem is not solved. No claim is made that an equivalent parameterization or special-case observation has never appeared elsewhere.

1.3 Current literature and the limits of this audit

Lackenby’s July 2026 preprint supplies substantial hierarchy machinery but explicitly separates an iteration analysis from a full running-time analysis [8]. A newer preprint by Musick, version 2 of September 2026, claims a polynomial-time algorithm through compressed locally minimal bridge presentations [9]. We do not assume or verify that claim here. In particular, the present article should not be read as a survey verdict that no stronger algorithm has been claimed. Its statements concern the specified algebraic route and the inspected ProveIt code.

Repository reads were pinned where possible. The adaptive-group synthesis was inspected at commit `f93328fbf5b5`, and the word protocol at `8170a64c7e72`. Full commit and blob identifiers are in `SOURCE_AUDIT.md`. No full upstream suite was run, and no upstream module was replaced. This is a self-contained research addition; `INTEGRATION.md` records the remaining adapter work.

2 Topological input and representation semantics

2.1 The two external topological facts

Throughout, a knot means a tame, one-component knot in S^3 . Crossing-free inputs are recognized directly and omitted from the group computations. Let G_K be the fundamental group of its complement. We use the following established facts [5, 6].

Theorem 2.1 (Representation input). *A nontrivial knot group admits a nonabelian representation in $SU(2)$. It also admits an irreducible meridian-traceless representation in $SU(2)$. Conversely, an unknot group admits neither kind of representation.*

The converse follows because the unknot group is $\mathbb{Z}$. For unitary two-dimensional representations, a reducible image preserves an orthogonal decomposition into two complex lines and is simultaneously diagonal; it is abelian. Conversely, an abelian collection of unitary matrices can be simultaneously diagonalized. Thus “irreducible” and “nonabelian image” are interchangeable here.

We require an independently verified identification between an input presentation and G_K . Merely checking its abelianization is not such an identification. The generic backend works after arbitrary verified presentation transformations. The special backend requires additional evidence that its two distinguished generators remain meridians, up to conjugacy and inversion.

2.2 Unit quaternions

Identify $SU(2)$ with the unit quaternions. Write $q = (a, b, c, d) = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$, with

$$a^2 + b^2 + c^2 + d^2 = 1.$$

Multiplication is

$$(a, b, c, d)(e, f, g, h) = (ae - bf - cg - dh, af + be + ch - dg, ag - bh + ce + df, ah + bg - cf + de).$$

The inverse of a unit quaternion is its conjugate $(a, -b, -c, -d)$. The corresponding matrix is

$$\begin{pmatrix} a + ib & c + id \\ -c + id & a - ib \end{pmatrix}.$$

It is unitary, not in general Hermitian. Its trace is $2a$, so meridian-traceless images are precisely pure imaginary unit quaternions.

Lemma 2.2 (A generator-independent noncommutativity test). *For unit quaternions $q_1, \dots, q_r$, write $v_i \in \mathbb{R}^3$ for their imaginary parts. Their generated group is nonabelian if and only if*

$$G(q_1, \dots, q_r) = \sum_{i < j} \|v_i \times v_j\|^2 > 0. \quad (1)$$

Proof. The imaginary part of $q_i q_j - q_j q_i$ is $2v_i \times v_j$, and its scalar part is zero. The sum in (1) vanishes exactly when every pair of generators commutes. Pairwise commuting generators generate an abelian group; the converse is immediate. $\square$

2.3 Two counterexamples that determine the API

The shortcut “two unequal generator images imply a nonabelian representation” is valid for the appropriate full Wirtinger situation, not for arbitrary transformed generators. Consider

$$\langle a, b \mid b = a^2 \rangle \cong \mathbb{Z}.$$

The unit quaternions $a = \mathbf{i}$, $b = -1$ are unequal and satisfy the relation, but commute. A generic backend must use (1), not inequality of images.

Nor may a change of generators preserve a “traceless” flag without a meridian proof. The trefoil has a presentation

$$\langle x, y \mid x^2 = y^3 \rangle. \quad (2)$$

If both generators were forced to be traceless, then $x^2 = -1$ and $y^3 = -y$, so the relation would force $y = 1$, which is not traceless. That restricted system is empty even though the knot is nontrivial. In contrast,

$$x = \mathbf{i}, \quad y = \frac{1}{2} + \frac{\sqrt{3}}{2}\mathbf{j}$$

satisfy $x^2 = y^3 = -1$ and have commutator difference $\sqrt{3}\mathbf{k} \neq 0$. This is an exact regression example, not a numerical observation.

These are *integration hazards*, not claims that the existing repository currently contains either bug. The current group stage does not implement this proposed feasibility backend. Our interface rejects a requested traceless specialization unless meridian provenance has been declared, and the integration contract requires verification of that provenance independently of the declaration.

3 A compressed presentation model

A straight-line program (SLP) is an acyclic word grammar. Node zero is the empty word. A terminal stores a signed generator. A product node stores the ordered pair of two earlier nodes. An inverse is represented by reversing products and negating terminals, with memoization. This is the protocol used by the inspected **WordArena** [2].

We represent a presentation as

$$\mathcal{P} = \langle g_1, \dots, g_r \mid U_\ell = V_\ell \ (1 \leq \ell \leq h) \rangle, \quad (3)$$

where both sides are SLP roots. Ordinary relators use $V_\ell = 1$. Equality pairs can lower the polynomial degree compared with first forming $U_\ell V_\ell^{-1}$.

Let s be the number of *live* nonempty nodes reachable from relation roots, and let N be the bit size of the complete encoded presentation, including generator count, node indices and root lists. Lifetime arena allocation is not s . Dead nodes left by unsuccessful searches must not inflate the measured algebraic parameter, and eliminated generators must not survive in reachable terminals.

The exporter checks root ranges, topological ordering and terminal names, rennumbers surviving generators, and copies only reachable nodes. It does not validate the producer’s Tietze transformations. A proof trace must already establish (3) as the group of the original validated knot diagram.

3.1 Polynomial checkpoints

Definition 3.1. A checkpoint set $\mathcal{C}$ is a subset of the live product nodes. Assign a unit quaternion variable q_i to each generator and four new real variables y_v to each $v \in \mathcal{C}$. Expand all other product nodes by quaternion multiplication, stopping when a checkpoint variable is encountered.

The defining equation of a checkpoint is retained:

$$y_v = E_v,$$

where E_v is its product expression in generator variables and earlier checkpoint variables. A checkpoint is not permission to forget this equation.

For degree accounting, terminals export degree one, the identity exports degree zero, and an uncheckpointed product exports the sum of its children's exported degrees. A checkpoint *defines* an expression of that same summed degree but exports degree one. Let

$$\Delta_C = \max\{2, \text{all checkpoint defining degrees, all relation-root exported degrees}\}. \quad (4)$$

This is a formal upper bound, not a claim that cancellation cannot lower an actual polynomial degree. It is computed in a DAG pass without expanding a word or polynomial. We write $c = |C|$ and

$$\kappa_C(\mathcal{P}) = (r + c) \log_2(\Delta_C + 2), \quad \kappa(\mathcal{P}) = \min_C \kappa_C(\mathcal{P}). \quad (5)$$

Finding the minimum on a shared DAG is not assumed to be easy. Every stated algorithmic guarantee is valid for the *chosen, explicitly provided* checkpoint set.

4 The complete degree-budgeted feasibility reduction

4.1 Construction and correctness

Form the following scalar polynomial residuals: the r generator norm equations; the four coordinate equations for each checkpoint; and the four coordinate differences for each relation $U_\ell = V_\ell$. Let $\mathcal{E}$ be this list and put

$$H = \sum_{f \in \mathcal{E}} f^2. \quad (6)$$

The feasibility query is

$$\exists z \in \mathbb{R}^{4(r+c)} : \quad H(z) = 0 \quad \wedge \quad G(z) > 0, \quad (7)$$

where G is (1) on the *generator* variables, not all auxiliary variables.

Theorem 4.1 (Exact compiler). *For any presentation (3) and checkpoint set, query (7) is feasible exactly when the presented group has a nonabelian $\text{SU}(2)$ representation. It has $4(r + c)$ real variables, two polynomial conditions, and degree at most $\max(2\Delta_C, 4) = 2\Delta_C$.*

Proof. Given a representation, assign its generator images and the values of all checkpoint words. Every residual vanishes, and nonabelianity gives $G > 0$ by Lemma 2.2.

Conversely, over the real numbers, a sum of squares is zero only when every summand is zero. The generator norm equations therefore put all generator images in $\text{SU}(2)$. Process the acyclic grammar in order. Each checkpoint is forced by its defining equations to equal the value of its represented word. All node values are unit quaternions by induction, so inverse terminals and memoized inverse grammars evaluate correctly by conjugation. No extra checkpoint norm equation is needed. Every original relation then holds, yielding a homomorphism from the presented group, and $G > 0$ makes it nonabelian.

A residual has degree at most Δ_C , with the norm equations covered by the minimum value two in (4). Squaring doubles the degree. The cross-product squares in G have degree four. $\square$

Remark 4.2. The argument is intrinsically real. Replacing real feasibility with existence of a complex zero of H would be wrong: over $\mathbb{C}$, a sum of squares can vanish without its individual summands vanishing. Likewise, a solver returning “unknown”, an unevaluated expression, or an approximate candidate has not answered (7).

4.2 Coefficient size and dense conversion

Lemma 4.3 (Coefficient envelope). *The integer coefficients of H and G have bit length*

$$O(\Delta_C + \log(r + c + h + 2)).$$

They can be explicitly constructed using

$$\text{poly}(N)(\Delta_C + 2)^{O(r+c)}$$

bit operations and space within the same envelope.

Proof. For a polynomial, use the sum of the absolute values of its coefficients. Each coordinate of a quaternion product is a sum of four products. A terminal coordinate or checkpoint variable has coefficient norm at most one. Inducting on an unfolded expression with formal degree d gives a coordinate norm bounded by a constant to the power d ; identity factors do not increase the norm. Subtraction of a root or checkpoint variable changes only the constant in this exponential estimate. Norm equations have fixed coefficient norm. Squaring residuals and adding $r + 4c + 4h$ of them gives coefficient bit length $O(\Delta + \log(r + c + h + 2))$. The polynomial G has at most quadratically many cross-product squares and obeys the same bound.

A polynomial of total degree at most 2Δ in $m = 4(r + c)$ variables has at most $(2\Delta + 1)^m$ monomials. Ordinary dense or sparse integer polynomial addition and multiplication, including monomial indices and coefficient arithmetic, are bounded by a fixed power of this number and the displayed coefficient bit length. Processing the live grammar and accumulating the residual squares adds a polynomial factor in N . This proves the claimed envelope. We are not treating an arithmetic circuit as if its dense expansion were free. $\square$

4.3 Bit complexity and the quasi-polynomial regimes

We use the singly exponential decision bound for the *existential* theory of the reals: a system of a fixed number of polynomial conditions of degree d in m variables, with integer coefficients of bit length τ , can be decided in $\text{poly}(\tau)(d + 1)^{O(m)}$ bit operations [4, 3]. This is not a bound for arbitrary quantifier alternation, nor automatically a performance guarantee for a computer algebra system's **Resolve** command.

Theorem 4.4 (Rank-checkpoint-degree bound). *A group presentation of bit size N , with r generators and an explicitly chosen checkpoint set $\mathcal{C}$, admits a deterministic decision procedure for nonabelian $\text{SU}(2)$ representability with bit complexity*

$$T_{\text{alg}}(\mathcal{P}, \mathcal{C}) \leq \text{poly}(N)(\Delta_C + 2)^{O(r+c)} = \text{poly}(N)2^{O(\kappa_C(\mathcal{P}))}. \quad (8)$$

If discovery and independent verification of this presentation and its knot-group provenance cost V , complete recognition costs at most $V + T_{\text{alg}}$.

Proof. Apply the existential-real decision bound to Theorem 4.1, using Lemma 4.3 to account for formula construction and coefficients. A polynomial factor in Δ is absorbed into $(\Delta + 2)^{O(r+c)}$, since $r \geq 1$. For a verified knot group, Theorem 2.1 converts feasible to nontrivial and infeasible to unknot. Adding the independent presentation cost gives the last assertion. $\square$

Three useful regimes follow immediately. With every live product checkpointed, $\Delta \leq 2$, giving $\text{poly}(N)2^{O(r+s)}$. With no checkpoints and maximum expanded side length L , the bound is $\text{poly}(N)(L + 2)^{O(r)}$. With an intermediate set, the relevant condition is exactly the mixed product in (5), not either endpoint separately.

Corollary 4.5 (Sufficient quasi-polynomial condition). *For n -crossing inputs, suppose a verified presentation of size $N = n^{O(1)}$ and checkpoints can be found and replayed in $2^{O(\log^2(n+2))}$ time, and*

$$(r + c) \log_2(\Delta_c + 2) = O(\log^2(n + 2)).$$

Then recognition of that input family has a $2^{O(\log^2(n+2))} = n^{O(\log n)}$ bound.

For example, $r + s = O(\log^2 n)$ suffices with full lifting, and $r = O(\log n)$ with $L = n^{O(1)}$ suffices without lifting. No theorem here forces either condition for every residual input in the maintained pipeline.

5 Why intermediate checkpointing is mathematically useful

5.1 A strict exponent separation

Proposition 5.1 (Hybrid separation). *There are word-presentation DAGs indexed by m for which both the no-checkpoint and all-checkpoint values of κ are $\Theta(m^2)$, while an explicit mixed set has $\kappa = O(m \log m)$.*

Proof. Use m generators $x_1, \dots, x_m$. Construct $P = x_1^{2^m}$ with m repeated-squaring products. Let W be the word obtained by concatenating $x_i x_j$ over all ordered pairs $(i, j) \in [m]^2$, in lexicographic order, and make $PW = 1$ the relation. The word W has length $2m^2$ and a balanced binary grammar of $O(m^2)$ nodes. Its m^2 distinct two-letter blocks ensure $\Omega(m^2)$ live product nodes, even after exact interning.

Without checkpoints the relation has degree $2^m + 2m^2$, so $\kappa = \Theta(m^2)$. With all product nodes checkpointed, there are $\Theta(m^2)$ new quaternion variables and degree two, again giving $\Theta(m^2)$. Instead checkpoint only the repeated-squaring chain. It has m nodes, each defining degree at most two; the remaining relation degree is at most $2m^2 + 1$. Consequently $\kappa \leq 2m \log_2(2m^2 + 3) = O(m \log m)$. $\square$

This is a separation for the compiler's cost parameter. These synthetic presentations are not asserted to be knot groups, to survive optimal group simplification, or to represent a hard-knot corpus. Their purpose is to show that the intermediate option is not merely cosmetic.

m	Live products	No checkpoints	All checkpoints	Mixed
4	35	22.58	78.00	40.70
8	135	68.74	286.00	112.36
16	527	256.18	1086.00	288.18
32	2079	1024.00	4222.00	704.09
64	8255	4096.00	16638.00	1664.05

The table reports κ for the exact implemented grammars. The asymptotic advantage need not appear at small m ; at $m = 16$ no substitution is still cheaper by this parameter. This negative small-size result is retained rather than obscured.

5.2 An optimal formula-tree algorithm

A formula tree has no shared subcomputations: two occurrences are separate even when their text is identical. For a fixed cap $D \geq 2$, let $F_v(d)$ be the minimum checkpoint count in the subtree of v , subject to all internal checkpoint equations having degree at most D and to exported degree $d \leq D$.

At a leaf, $F_v(1) = 0$. At a product with children u, w , each pair a, b with $a + b \leq D$ gives two options:

$$F_v(a + b) \leq F_u(a) + F_w(b), \quad (9)$$

$$F_v(1) \leq F_u(a) + F_w(b) + 1. \quad (10)$$

The second option checkpoints the product. It is still required that its defining expression have degree $a + b \leq D$.

Theorem 5.2 (Optimal bounded-degree checkpoints on a tree). *For a formula tree with s occurrences, recurrences (9)–(10) find a minimum-cardinality checkpoint set under degree cap D in $O(sD^2)$ arithmetic operations and $O(sD)$ records of working storage. A selected set can be reconstructed with backpointers.*

Proof. Every legal choice either checkpoints the root or does not. In both cases its two subtrees are independent and their exported degrees determine the root defining degree. These are exactly the two recurrences, so induction proves optimality. Each node combines at most D^2 degree pairs. Store the cost and child-degree choices, not a copied full checkpoint list, at every entry. This yields the storage bound and a linear-size reconstruction pass. $\square$

The state-space dependence is polynomial in the *numerical* cap D , not in $\log D$. Costs are integers of $O(\log s)$ bits. On a shared DAG, independent subtree costs can count the same checkpoint repeatedly and can choose inconsistent sharing decisions. The tree theorem must not be applied to that problem. The package includes exhaustive optimization only for small shared DAGs and a separately labeled feasible greedy rule for larger DAGs.

6 Optimal fixed-seed Wirtinger propagation

6.1 Rules and exact degree labels

An oriented crossing is recorded as (o, u, v, ε) with relation

$$v = o^\varepsilon u o^{-\varepsilon}, \quad \varepsilon \in \{1, -1\}. \quad (11)$$

Arcs are generators between consecutive undercrossings. Choose a nonempty set S of seed arcs and give each seed formal degree one. The allowed derivations use (11) in either direction. Their unreduced word lengths satisfy the rules

$$(u, o) \longrightarrow v : d_v = d_u + 2d_o, \quad (v, o) \longrightarrow u : d_u = d_v + 2d_o. \quad (12)$$

The sign affects the word expression but not this degree cost.

Definition 6.1. The minimum fixed-seed degree $d_S(a)$ is the smallest unreduced word length of a derivation of arc a using (12), or infinity if there is no such derivation. It is not the length of a shortest freely reduced word and not the optimum over seed sets.

The rules form a directed hypergraph with two prerequisites per rule. Duplicate prerequisites are tracked once for readiness, while their weights remain one and two. Initialize the seeds at one and process a priority queue in increasing tentative degree. A rule is activated once both its prerequisites have been settled. It proposes their weighted sum for its target.

Theorem 6.2 (Minimum formal degree). *The resulting generalized Dijkstra procedure computes all $d_S(a)$. For n arcs and h crossings it uses $O((n + h) \log(n + h + 2))$ arithmetic/queue operations. Every finite label has $O(n)$ bits. For a diagram with $h = O(n)$, a conservative bit bound is $O(n^2 \log(n + 2))$.*

Proof. All finite labels are positive. A rule's output is strictly larger than either input. Consider a minimum-cost derivation of an arc. Its two prerequisite derivations have strictly smaller costs. Thus, if a queue extraction were the first incorrect settled label, the prerequisites of an optimal derivation for that arc would already have been settled correctly and would have proposed its optimal value, a contradiction. Unreachable arcs remain infinite because a finite proposal always has a derivation.

There are two rules per crossing. Each is activated at most once, so there are $O(n + h)$ queue entries and the stated operation bound. A settled nonseed label is at most three times the largest earlier settled label. Starting with one and settling at most n arcs bounds every label by 3^n . Consequently integer additions and comparisons cost $O(n)$ bit operations, yielding the conservative diagram bound. $\square$

6.2 A checkable optimality certificate

The certificate contains seeds, finite labels, a settlement order and, for each derived arc, its chosen crossing and direction. A checker verifies that every chosen rule uses earlier arcs and attains its label exactly. It also checks every enabled rule's Bellman inequality

$$d_{\text{target}} \leq d_{\text{source}} + 2d_{\text{over}},$$

and rejects an infinite target with two finite prerequisites.

The attainment witnesses prove each finite label is achievable. Conversely, induction on any derivation and the Bellman inequalities proves the assigned label is no greater than that derivation's cost. Reachability closure excludes a missed finite derivation. Hence these checks certify optimality without trusting the priority-queue implementation.

Proposition 6.3 (Presentation preservation under seed substitution). *If all arcs are derived, substituting their certified seed words into every original crossing relation gives a presentation of the same group on $|S|$ generators. The generators remain the chosen original meridians. The word DAG has $O(n + h)$ nodes, with memoized inverse nodes included.*

Proof. For each nonseed arc, its chosen relation has that arc as the newly defined generator and has only earlier derived arcs on the other side. It is therefore a valid successive elimination. Induction expresses every eliminated generator as its recorded seed word. Substituting in all original relations gives precisely the quotient induced by these eliminations; defining relations may become identities, which are harmless. Equivalently, the maps sending each original arc to its seed word and each seed to its original arc are inverse on the resulting quotients.

Each derivation needs a constant number of products, and each original relation needs a constant number more. Memoized inversion creates at most one reversed/negated copy of each existing reachable node. Thus the shared grammar remains linear in the number of derivations and relations. No expanded substitution is required. $\square$

6.3 Visible overpasses give a concrete quasi-polynomial class

Let b be the number of distinct Wirtinger arcs used as the over-arc of at least one crossing. These are the visible overpasses of the *supplied* diagram, not a minimized topological bridge number.

Theorem 6.4 (Visible-overpass bound). *For an n -crossing one-component knot diagram with b visible overpasses, the group-representation decision procedure has bit complexity $n^{O(b)}$ (with $n + 2$ in place of n for small inputs). In particular, $b = O(\log(n + 2))$ gives $2^{O(\log^2(n+2))}$ recognition.*

Proof. Take every over-arc as a seed. Starting at a seed and following the oriented knot, every undercrossing has a known over-arc and derives the next arc. Thus all arcs are reached. Each over-arc has degree one, so each step increases degree by at most two; all arc degrees are at most $2n + 1$. The two sides of each substituted crossing equality have degree at most $2n + 3$. The grammar size is $O(n)$ by Proposition 6.3. Apply Theorem 4.4 with $r = b$, no checkpoints, and $\Delta \leq 2n + 3$. Construction and replay are polynomial. $\square$

This theorem is about a visible, checkable diagram parameter. A short description of a bridge presentation does not by itself bound the expanded word degree used by this proof. In particular, no efficient conversion from an arbitrary diagram to a logarithmic-overpass diagram is asserted.

7 Two meridians: from real algebra to integer arithmetic

7.1 A complete gauge for the nonabelian traceless slice

Suppose a presentation has exactly two *certified meridian* generators a, b . In any nonabelian meridian-traceless representation their images A, B are nonparallel pure imaginary unit quaternions. Simultaneous conjugation acts by rotations of their imaginary vectors. Rotate the first vector to $\mathbf{i}$, then rotate around that axis to put the second in the positive $\mathbf{j}$ half-plane. Thus every such representation is conjugate to exactly one pair of the form

$$A = \mathbf{i}, \quad B = -\cos \phi \mathbf{i} + \sin \phi \mathbf{j}, \quad 0 < \phi < \pi. \quad (13)$$

The parameter is determined by the scalar product. The two excluded endpoints are exactly the commuting pairs.

Put $T = AB = \cos \phi + \sin \phi \mathbf{k}$. We have

$$A^2 = B^2 = -1, \quad ATA^{-1} = T^{-1}, \quad B = -T^{-1}A. \quad (14)$$

These are the familiar binary-dihedral relations. Their use here is as a small exact target for a compressed word compiler.

7.2 The three-entry normal form

Lemma 7.1 (Compressed normal form). *Every word in $a^{\pm 1}, b^{\pm 1}$ evaluates, for every pair (13), as*

$$(-1)^e T^k A^s, \quad (e, k, s) \in \{0, 1\} \times \mathbb{Z} \times \{0, 1\}. \quad (15)$$

One can compute such a triple by assigning

$$\begin{aligned} a &\mapsto (0, 0, 1), & a^{-1} &\mapsto (1, 0, 1), \\ b &\mapsto (1, -1, 1), & b^{-1} &\mapsto (0, -1, 1), \end{aligned}$$

and using

$$(e, k, s)(f, \ell, t) = (e + f + st \bmod 2, k + (-1)^s \ell, s + t \bmod 2). \quad (16)$$

The inverse triple is $(e, -k, 0)$ when $s = 0$, and $(e + 1 \bmod 2, k, 1)$ when $s = 1$.

Proof. The terminal formulas follow from (14). Moving A^s past T^ℓ replaces ℓ by $(-1)^s \ell$, and multiplying two remaining A factors contributes $A^2 = -1$. This proves (16). The inverse formulas follow by multiplication. Structural induction on an SLP proves the claim for every represented word, regardless of its expanded length. $\square$

For a relation $U_j = V_j$, compute the triple of $U_j V_j^{-1}$ and call it (e_j, k_j, s_j) . If $s_j = 1$, the image is pure imaginary in the $\mathbf{i}, \mathbf{j}$ plane, so it cannot equal one. Otherwise its value is one exactly when

$$k_j \phi \equiv e_j \pi \pmod{2\pi}. \quad (17)$$

An equation with $k_j = 0, e_j = 1$ is also immediately impossible.

7.3 The complete phase test

Theorem 7.2 (Integer feasibility criterion). *After the two immediate obstructions just described have been excluded, put*

$$d = \gcd_j |k_j|.$$

If $d = 0$, every $0 < \phi < \pi$ satisfies the relations. If $d > 0$, choose an index with k_j/d odd and set $e = e_j$. The relations are consistent on the nonabelian slice exactly when

$$e_j \equiv (k_j/d)e \pmod{2} \quad \text{for every } j \quad (18)$$

and

$$\nu = \left\lfloor \frac{d-1+e}{2} \right\rfloor > 0. \quad (19)$$

When consistent, there are exactly ν conjugacy classes in this traceless, noncommuting two-generator slice. A compact exact witness is

$$\phi = \frac{\pi j}{d}, \quad j = \begin{cases} 1, & e = 1, \\ 2, & e = 0, \end{cases}$$

provided $j < d$.

Proof. If $d = 0$, all relations are the identity normal form, so there is no restriction. Suppose $d > 0$ and write $a_j = k_j/d$. Their greatest common divisor is one, so at least one is odd.

If a solution exists, put $Z = e^{id\phi}$ in the usual complex unit circle. Equations (17) say $Z^{a_j} = (-1)^{e_j}$. By an integer Bezout combination of the a_j , Z must itself be 1 or -1 . Write it as $(-1)^e$. At an odd a_j , this determines $e = e_j$, and all other equations force (18).

Conversely, if (18) holds, the entire relation system is equivalent to $e^{id\phi} = (-1)^e$. Its solutions in $0 < \phi < \pi$ are exactly $\phi = \pi j/d$ with integers $0 < j < d$ and $j \equiv e \pmod{2}$. Counting those integers gives (19), and the smallest admissible numerator is the one displayed. The gauge parameter uniquely specifies the conjugacy class of the ordered noncommuting pair, so the same number counts classes in this slice. $\square$

The proof justifies an exact symbolic witness without constructing an algebraic number of potentially huge degree. A verifier checks the rational phase numerator and denominator, the strict inequalities, and the integer congruences. Existence of the corresponding unit quaternions is the elementary trigonometric construction (13). No approximation to $\cos(\pi j/d)$ is necessary.

7.4 Compressed bit complexity

Theorem 7.3 (Polynomial two-meridian backend). *Let a two-meridian presentation have s live SLP nodes, h relations, and encoded bit size N . Nonabelian meridian-traceless $\text{SU}(2)$ feasibility is decidable in time polynomial in N , without expanding words or polynomials. More explicitly, if all intermediate exponents have at most B bits, an elementary bound is*

$$O(N + sB + hB^3) \quad (20)$$

bit operations, apart from inessential encoding conventions. One may take $B = O(s + 1)$. The normal forms and the phase criterion give replayable certificates for either answer. For a verified knot-group presentation with the stated meridian provenance, the answers completely decide unknot recognition.

Proof. Each grammar product uses a bounded number of additions, sign changes and parity operations. A terminal exponent has absolute value at most one; under a product it grows by at most the sum of the two input magnitudes. Thus an exponent is bounded by represented word length. In a topologically ordered binary SLP with s nodes that length is at most $2^{O(s)}$, and a relation comparison changes this only by a factor of two. Hence $B = O(s + 1)$.

The grammar pass costs $O(sB)$ bit operations. Computing a gcd of h B -bit integers by the Euclidean algorithm is polynomial. For an elementary conservative bound, there are $O(B)$ divisions per gcd and schoolbook division costs $O(B^2)$, giving $O(hB^3)$. Quotients and parity checks fit this bound. Reading the encoded input adds $O(N)$. Recording triples uses $O((s + h)B)$ bits. The phase test is exact by Theorem 7.2; a checker can recompute the normal forms and gcd from the original grammar rather than trust supplied exponents. Finally apply Theorem 2.1 and the complete gauge (13). $\square$

The word-size conclusion is stronger than simply applying a fixed-dimensional real solver after expansion. For an SLP representing length 2^s , the latter can still take $2^{O(s)}$ time. The two-meridian normal-form target removes that expansion from the computation. This does not extend automatically to three meridians: three independent imaginary axes need not lie in the same plane, so the binary-dihedral normal form no longer contains every representation.

8 A polynomial diagram-level certificate class

Call a validated knot diagram *two-seed derivable* if all its Wirtinger arcs can be obtained from at most two seed arcs by (12). This is a property of the supplied diagram and its allowed derivations, not an assertion about a minimized bridge diagram.

Theorem 8.1 (Complete recognition on two-seed derivable diagrams). *There is a polynomial-time, sound partial unknot recognizer on all diagrams that is conclusive on every two-seed derivable diagram. With an exhaustive seed search and elementary arithmetic, a conservative bound on this restricted class is $O(n^4 \log(n + 2))$ bit operations for n -crossing diagrams.*

Proof. Try every one-element and then every two-element seed set. There are $O(n^2)$ choices. Theorem 6.2 finds a successful set, when one exists, within $O(n^2 \log(n + 2))$ bit operations per choice. The resulting derivation certificate is replayed and Proposition 6.3 produces an $O(n)$ -node presentation on the seed meridians.

With one seed, the verified knot group is cyclic and the knot is the unknot. With two seeds, apply Theorem 7.3. Here $s, h, B = O(n)$, so the coarse arithmetic bound is $O(n^4)$. The seed search dominates only by a logarithmic factor. Both answers are conclusive on the restricted class. If all seed choices fail, return no conclusion; a larger meridional generating set may exist and the original complete fallback remains available. $\square$

This is a structural certificate class, not a general recognition theorem. The mechanism can be installed before the expensive homology backend, but whether it finds certificates missed by existing simplification stages is an empirical question not settled by the presentation benchmarks below. Fixed small seed search was not claimed here to minimize the topological bridge number.

9 An independent univariate formulation

The integer backend is the preferred two-meridian algorithm. A distinct univariate derivation is valuable as an independent mathematical check and as a reference implementation that uses ordinary exact real algebra.

9.1 Rational parameterization

Set

$$A = (0, 1, 0, 0), \quad B(t) = \left(0, \frac{1-t^2}{1+t^2}, \frac{2t}{1+t^2}, 0\right), \quad t > 0. \quad (21)$$

This is the same gauge as (13), with $t = \cot(\phi/2)$. Since $1+t^2 > 0$, clearing its powers introduces no positive real solutions and removes none.

For each word node maintain a numerator quaternion in $\mathbb{Z}[t]^4$ and a denominator $(1+t^2)^e$, where e is the number of occurrences of $b^{\pm 1}$. A negative terminal uses $-A$ or $-B$, because these images are traceless. Multiplication adds denominator exponents and multiplies quaternion numerators. For a relation $U = V$, clear to the larger denominator exponent, rather than multiplying both denominators unnecessarily.

If a represented side has at most L letters, its numerator degree is at most $2L$ and its coefficients have $O(L)$ bits. Repeated squaring can make this bound exponential in the SLP size. Therefore the implementation preflights represented b -degree before allocating coefficient arrays and reports UNKNOWN when the cap is exceeded.

9.2 Gcd and Sturm correctness

Let $f_1, \dots, f_q \in \mathbb{Z}[t]$ be all scalar residual polynomials. If they all vanish identically, every $t > 0$ is feasible. Otherwise let g be their primitive gcd over $\mathbb{Q}[t]$.

Proposition 9.1 (Exact univariate decision). *The two-generator noncommuting traceless slice is nonempty if and only if g has a positive real root, with the all-zero convention above. This can be decided by exact gcd and Sturm calculations, with no numerical root tolerance.*

Proof. A real number is a common root of the residuals exactly when it is a root of their gcd. One direction follows because g divides every residual; the other follows from the Bezout identity for the ideal they generate in $\mathbb{Q}[t]$. The positive denominator in (21) makes residual vanishing equivalent to the original group relations. The parameter range is exactly the noncommuting range of the gauge.

Remove all factors of t from g because zero is excluded. Divide by the gcd with its derivative to obtain its square-free part. For that polynomial, the Sturm chain counts distinct roots in $(0, +\infty)$ as $V(0) - V(+\infty)$, omitting zero signs when counting sign variations. At zero use the constant coefficient; at positive infinity use the leading coefficient. Exact rational remainders give the signs without approximations. Scaling a chain polynomial by a positive rational is harmless; forcing every leading coefficient positive is *not* harmless and is not done. $\square$

The implementation uses reduced rational arithmetic and primitive remainders. Its role is a small-instance reference, not a claim of optimal subresultant performance. The compressed integer backend has the stronger input-size guarantee and avoids the numerical degree dependence entirely.

9.3 Examples and exact cross-checks

For the trefoil relation $aba = bab$, the nonzero residual numerators in one clearing convention are

$$2t^2(t^2 - 3), \quad -2t(t^2 - 3).$$

Their primitive gcd is $t(t^2 - 3)$, giving the unique positive solution $t = \sqrt{3}$. The integer backend gives $(e, d) = (1, 3)$ and $\phi = \pi/3$, hence $t = \cot(\pi/6) = \sqrt{3}$.

For the figure-eight fixture, we start with the explicit three-braid $(\sigma_1\sigma_2^{-1})^2$, construct its crossing relations, and replay propagation from seed arcs 0, 1. The minimum labels are $(1, 1, 5, 3)$. The resulting univariate gcd is

$$t(t^4 - 10t^2 + 5),$$

with two positive roots. The integer backend has $d = 5, e = 1$ and two phases, $\pi/5$ and $3\pi/5$. This figure-eight computation does not distinguish the trefoil from other knots with the same traceless binary-dihedral counts; the purpose of the backend is unknot recognition, not complete knot classification.

The separate Wolfram computation returned the exact trefoil solution, rejected $a = b$ on the noncommuting slice, rejected the unsafe traceless specialization of (2), and verified the explicit nonmeridional witness in Section 2. The service’s warning about global symbol names is preserved alongside its evaluated results in `data/wolfram_check.txt`; it was not discarded from the audit trail.

10 Proof-carrying integration and failure semantics

10.1 The trust boundary

The public algebraic status words are **EXISTS**, **NONE** and **UNKNOWN**. They mean that a specified representation slice exists, does not exist, or has not been decided. They are deliberately not named “UNKNOT” or “KNOTTED”. A caller may convert them to topology only after checking the appropriate input contract:

1. The original input is a valid, one-component classical knot diagram, and its crossing or group data were reconstructed from that exact input.
2. Every presentation transformation or seed derivation is independently replayed. All remaining original relations are retained under substitution, including relations not selected for deriving generators.
3. The generic compiler uses actual commutators. The two-meridian branch additionally has a proof that the two generators are original meridians or their certified conjugates/inverses.

The `meridian_generators` field is an explicit precondition declaration in a standalone research file, not a cryptographic or mathematical certificate of meridiancy. In particular, an arbitrary user-edited JSON flag is never a sufficient reason to accept a knot verdict.

For fixed-seed substitution, meridian provenance is simple: the remaining generators are untouched original seed arcs. After a Whitehead or other generator-changing move, this inference is no longer valid. A production adapter should default to the generic backend unless explicit meridian provenance has been preserved and checked.

10.2 Replayable algebraic evidence

The integer certificate records the presentation digest, the normal form of each comparison relator, the gcd and phase parity when relevant, and either a compact rational phase or a specific

obstruction. The verifier recomputes from the original SLP. For the univariate method, the record contains the source digest, residuals, gcd, square-free positive-domain polynomial and Sturm signs; replay also starts from the source presentation, not the supplied polynomial list.

These verifiers share their arithmetic kernels with their producers. They are not advertised as independent proof-assistant checkers. Independence is instead provided by the separate exact matrix compiler, SymPy real-root comparisons, rational-quaternion evaluations and a Wolfram regression calculation. A future Lean checker can replace this software trust boundary without changing the mathematical certificate format.

10.3 Budget accounting

Search and replay are separate operations. The adapter must preserve the original presentation-search budget, and a change of representation must not reset a deadline or work counter. Resource ceilings remain reasons for UNKNOWN, never for NONE. A configured compiler may reject an excessive formal degree before expansion; this only means that its current representation choice is too expensive. Another checkpoint set or the integer specialization may still finish.

The generic package component is a formula compiler and a Wolfram-language query exporter. It does not implement Renegar’s algorithm and does not establish that Wolfram’s chosen internal method satisfies (8). A proof-carrying production use of arbitrary-rank NONE answers still needs an accepted exact solver/checker interface. In contrast, the two-meridian integer routine already decides its full specified algebraic slice without an external solver.

11 Implementation, validation, and measured performance

11.1 Delivered implementation

The Python package `su2budget` has no runtime dependency outside the standard library. The modules are intentionally separated by mathematical responsibility:

Module	Responsibility
<code>slp.py</code>	Validated word DAGs, reachable-node export, identities and inverses.
<code>degrees.py</code>	Exact formal degrees; small-DAG exhaustive selection; feasible greedy selection; optimal formula-tree dynamic programming.
<code>multivariate.py</code>	Capped sparse integer polynomial compiler, true commutator obstruction, and exact Wolfram query export.
<code>wirtinger.py</code>	Fixed-seed minimum degrees, independent inequality/witness replay, all-relation seed substitution, and bounded one/two-seed search.
<code>dihedral.py</code>	Compressed integer normal forms, complete phase criterion, compact certificates and replay.
<code>univariate.py</code>	Separate rational parameterization, exact polynomial gcd and Sturm reference implementation.

The package also contains literal braid fixtures, an independent SymPy matrix compiler, batch timing scripts, all raw timing samples and generated article tables. No upstream files were overwritten.

11.2 Completed validation

All **30 unit tests passed** on CPython 3.13.5. These are test methods, not a claim that the full ProveIt suite was run. Their internal checks include:

- 8,000 exhaustive associativity comparisons of the integer triple product; inverse checks; and 640 exact rational-quaternion evaluations of independently formed words and normal forms.
- 300 seeded synthetic group presentations compared between the integer and univariate backends, with the number of positive solutions compared whenever supplied by the phase criterion. Synthetic groups are not claimed to be knot groups.
- 400 random fixed-seed hypergraphs compared with independent repeated relaxation, with certificate replay; exhaustive formula-tree checkpoint comparisons for all binary tree shapes with at most six leaves and caps two through six.
- Known knot and unknot braid cases, retained crossing-relation checks, certificate tampering, live-node export, malformed grammars, caps returning UNKNOWN, forbidden nonmeridional specializations, and sparse compiler evaluation against exact quaternion values.

A separate SymPy 1.14.0 program independently multiplied 2×2 complex matrices for **115 presentations**: 100 seeded synthetic systems and 15 two-strand torus relations. Its exact positive-root answers agreed with the integer algorithm throughout. It also compared **200** independent univariate real-root counts with our Sturm routine. The literal matrix construction does not use the quaternion polynomial compiler. Complete inputs and outcomes are in `data/independent_validation.json`.

These tests support implementation correctness. They are not substitutes for the proofs, an audit of all possible upstream transformations, or a benchmark of unknown hard knots.

11.3 Paired exact presentation queries

We compare the integer backend with the univariate reference on the meridional presentations

$$\mathcal{P}_m = \langle a, b \mid (ab)^m a = b(ab)^m \rangle, \quad p = 2m + 1. \quad (22)$$

They are the standard $(2, p)$ torus-knot presentations. These are easy mathematical knots, not evidence of a new hard-knot capacity for the whole recognizer.

For every recorded trial, we validate a fresh presentation object, solve it and replay the resulting certificate. Presentation construction is outside the timed interval. There are five shuffled rounds, with each arm averaging 20 queries. An identical integer control arm exposes scheduling and timing variation. The table gives medians of these batch averages; every raw sample is retained.

p	Integer (ms)	Control (ms)	Univariate (ms)	Ratio
3	0.0292	0.0289	0.658	22.5
5	0.0326	0.0406	1.217	37.3
9	0.0393	0.0424	2.346	59.7
17	0.0425	0.0450	5.360	126.2
33	0.0442	0.0449	15.070	341.0
65	0.0434	0.0483	49.167	1132.8

The integer method avoids dense polynomial expansion and root counting, so its advantage over this reference increases with p . The large ratios are *not* speedups over **fastunknot**. The control variation is material at tens of microseconds, and this small, deliberately structured corpus does not establish general performance. The case for default pipeline installation must come from a separate paired whole-recognizer study, particularly because existing Alexander/Jones or braid stages can already decide these examples.

11.4 Capacity with exponentially long represented words

Let $m = 2^b$ in (22), and construct $(ab)^m$ by b repeated squarings. Its expanded word length is exponential in b , while the grammar is linear in b . Direct calculation from (16) gives the comparison relator normal form

$$(1, 2m + 1, 0).$$

Thus $d = 2m + 1$, $e = 1$, and there are m noncommuting traceless classes. The solver stores this count in binary and does not enumerate classes.

Squaring depth b	Live nodes	Exponent bits	Solve + replay (ms)
64	69	66	0.129
256	261	258	0.397
1,024	1,029	1,026	1.453
4,096	4,101	4,098	6.052
10,000	10,005	10,002	16.412

The measurements include solve and replay on an already constructed presentation, with three samples per size. At depth 10,000 the input has 10,005 live nodes and represents the torus parameter $2^{10001} + 1$. This is a *compressed-presentation* experiment, not a claim to have read or recognized an explicit diagram with that many crossings. The exact answer is supported by the normal-form proof and integer certificate, not by extrapolated timing.

12 What this changes about the quasi-polynomial program

The former group-stage endpoint was particularly restrictive: find a verified cyclic presentation or remain inconclusive. The generic theorem provides a different endpoint: find a verified presentation whose rank, checkpoint count and exposed degree have small combined cost. That can decide either side of recognition. The two-meridian specialization makes one such endpoint cheap enough to implement with a small exact arithmetic kernel.

There are two distinct ways to exploit the result. A diagram may have a visible structural certificate, such as a small successful seed set, whose discovery and verification are polynomial. Alternatively, a more powerful group simplifier may find a low-cost presentation after transformations. The second route needs separate search bounds and careful meridian tracking. Our compiler does not make an incomplete presentation heuristic complete by itself.

The rank–degree accounting clarifies why several plausible shortcuts fail. A bounded number of generators with enormous unexpanded degree is not automatically a fast generic real query. A bounded grammar with too many generators is not automatically a quasi-polynomial query. Checkpointing every node can replace a degree problem by a dimension problem. A small parameter certificate that takes uncontrolled time to discover does not give an end-to-end bound. Finally, a fast one-sided obstruction does not give an unknot certificate when it fails.

The explicit sufficient target for this route is

$$\log V + \log(N + 2) + (r + c) \log(\Delta + 2) = O(\log^2(n + 2)), \quad (23)$$

with a harmless convention for $V = 0$. One may replace the squared logarithm by a larger fixed power for a broader quasi-polynomial target. Equation (23) is a research specification, not a proved universal invariant of diagram simplification.

The exact two-meridian backend also exposes a useful lesson about representation choice: the appropriate target algebra can matter more than generic elimination order. A grammar that

looks expensive as a polynomial circuit may be cheap in a normal-form algebra tailored to a complete representation slice. Finding similarly complete small targets beyond two meridians is a mathematical question, not just an engineering optimization.

13 Further research questions and proposed work

- Q1. Presentation-search bounds for residual hard inputs.** After the current inexpensive certificates have been exhausted, can one prove that a presentation satisfying (23) can be found within the same budget? A theorem must bound the search and replay, not just exhibit a short presentation nonconstructively. Begin with explicitly defined families not already settled by the existing filters.
- Q2. Meridian-preserving simplification.** Which existing generator eliminations can be reordered to retain a small set of original meridians while keeping the word DAG small? Compare this constrained search with unrestricted Whitehead reduction. The smallest ordinary generator rank need not be the best rank for the two-meridian specialization.
- Q3. Replayable meridian witnesses after generator changes.** Design a certificate that a surviving generator is conjugate to an original meridian or its inverse, using the existing compressed-word proof trace. Exponent sum ± 1 alone is insufficient. A checker should verify an actual conjugacy identity with group-theoretic provenance.
- Q4. Three-meridian complete target algebras.** Two pure axes lie in a common plane; three generally do not. Can the complete three-meridian traceless representation problem be expressed using a small set of Gram coordinates and an orientation parameter while preserving efficient compressed evaluation? The challenge is to avoid reintroducing exponential polynomial degree in the new coordinates.
- Q5. Optimal checkpointing on shared DAGs.** Determine the complexity of minimizing κ_C or the checkpoint count at a fixed degree cap with shared subexpressions. Prove approximation guarantees for practical algorithms, or derive fixed-parameter algorithms in a structural width of the DAG. The formula-tree theorem does not answer this problem.
- Q6. Sharper degree certificates.** Formal degree is safe but can be very pessimistic after norm identities, inverse cancellation, or a trace relation. Develop an efficiently checkable upper-degree certificate that incorporates such reductions without assuming polynomial identity testing for an unrestricted circuit. Measure whether it changes the chosen checkpoint sets on real presentation traces.
- Q7. Compressed arithmetic beyond the binary-dihedral slice.** Look for additional normal-form targets that retain all representations needed for a complete decision on a specified family. Finite targets that provide only obstructions are not enough for negative feasibility to certify an unknot. Any completeness theorem must be stated with its exact family and peripheral hypotheses.
- Q8. More efficient seed-set discovery.** The fixed-seed optimum is solved here; global minimum seed selection is not. Can one find logarithmic-size seed sets with controlled formal degree on useful diagram classes, or certify that a searched class has no smaller set? Compare degree-aware selection with Boolean closure selection and with the repository's scan-order heuristics.
- Q9. Proof-assistant verification of the small kernel.** Formalize the triple product, gauge coverage, parity criterion and its topological interface. The arithmetic portion has a much smaller trusted base than general real elimination. The topological representation

theorem should remain an explicit imported theorem, not an unmarked axiom in an executable recognizer.

- Q10. Certified arbitrary-rank real feasibility.** Implement or connect an exact existential-real solver with a clearly documented bit bound and checkable results. A successful numerical search can guide an exact witness but cannot establish infeasibility. Study the size and verification cost of negative certificates separately from the decision bound.
- Q11. General generators with a marked meridian word.** A rank-two presentation may have nonmeridional generators, as in (2), while a meridian is a known compressed word in them. Add a traceless constraint on that word rather than on both generators, and analyze its checkpoint cost. This could preserve more of the power of unrestricted group simplification.
- Q12. Whole-pipeline ablations.** Install the two-seed search and the verified compressed endpoint behind opt-in switches. Benchmark complete recognition, including original PD validation, source reduction, certificate discovery, replay, failed searches and fallbacks. Record regressions and cases already settled by older filters rather than reporting only successful algebra kernels.
- Q13. Hierarchy-to-presentation transfer.** Investigate whether bounded hierarchy encodings can produce a low-cost group presentation in the sense of (5). A useful result must control peripheral data, correction steps and grammar growth. Do not assume that a logarithmic hierarchy depth by itself bounds generator rank or exposed polynomial degree.
- Q14. Audit of newer compressed bridge claims.** Independently replay the transformations and bit-size invariants in the September 2026 bridge-presentation claim [9]. Determine whether its encodings can provide certified small seed presentations to this backend. This is a proposed audit, not a conclusion about the correctness of that preprint.

14 Conclusion

The strongest implemented result is an exact, polynomial compressed-input decision for the complete noncommuting traceless slice on two certified meridians. Its certificates use only word-DAG evaluation, integer gcds, parity and a rational phase. Fixed-seed propagation connects that arithmetic to a checkable class of knot diagrams. For larger presentations, checkpointing yields a precise complete-algorithm bound and demonstrates that mixed dimension–degree choices can improve the asymptotic exponent over both naive extremes.

What remains for a general quasi-polynomial recognizer is an end-to-end mechanism producing sufficiently cheap certified inputs, or a different complete representation target that avoids their growth. The supplied package makes that boundary explicit, provides concrete exact code on one side of it, and supplies a cost specification and research questions for the other.

A Reproduction and artifact map

From the package root:

```
PYTHONPATH=code python -m unittest discover -s tests -v
PYTHONPATH=code python experiments/validate_independent.py
PYTHONPATH=code python experiments/benchmark.py
PYTHONPATH=code python experiments/make_artifacts.py
sh build.sh
```

Only the independent validation script requires SymPy. The article uses a standard LaTeX installation. Regenerating measurements replaces raw timing files with the new machine's samples; the committed results describe the recorded environment, not a promised runtime on another system.

Example commands:

```
PYTHONPATH=code python -m su2budget two-meridians examples/trefoil.json
PYTHONPATH=code python -m su2budget univariate examples/figure_eight.json
PYTHONPATH=code python -m su2budget profile examples/trefoil_nonmeridional.json
PYTHONPATH=code python -m su2budget wolfram examples/trefoil_nonmeridional.json
```

The first two commands return representation-feasibility records. They do not bypass the upstream knot-provenance contract. The last command emits an exact real formula; emitting it is not solving it.

Artifact	Contents
article.tex, pdf	The article and compiled PDF.
code/su2budget/	Standard-library reference implementation.
tests/	Thirty test methods with exhaustive and seeded subchecks.
experiments/	Independent matrix validation, paired timings and table/example generation.
examples/	Source presentations, replayable algebraic certificates, a figure-eight seed certificate and generic Wolfram queries.
data/	Raw tests, all timing samples, symbolic validation records and article tables.
docs/wolfram_check.wl	The exact independent Wolfram regression input.
INTEGRATION.md	Preconditions, recommended call sites and unimplemented production adapter work.
SOURCE_AUDIT.md	Inspected sources, pinned repository references and limits of verification.
CHECKSUMS.sha256	Integrity hashes for the delivered files.

B Deriving the torus presentation used in capacity tests

For $m \geq 1$, write $c = ab$ and $h = c^m a$ in (22). Its relation implies

$$ac^m a = c^{m+1}, \quad h^2 = c^m ac^m a = c^{2m+1}.$$

Conversely, from $\langle h, c \mid h^2 = c^{2m+1} \rangle$ recover

$$a = c^{-m} h, \quad b = h^{-1} c^{m+1}.$$

These expressions give inverse presentation transformations and recover the original relation. This is the usual two-generator torus-group presentation, while a, b are the meridional generators in the standard odd two-strand braid presentation. The capacity test retains those meridional generators; it does not incorrectly force the nonmeridional h, c to be traceless.

In the normal form, $(ab)^m a$ has triple $(0, m, 1)$ and $b(ab)^m$ has triple $(1, -m - 1, 1)$. Their comparison has triple $(1, 2m + 1, 0)$. This proves the exact gcd and phase count used to validate the huge compressed examples without appealing to measured timings.

C A compact independent-checking specification

A minimal integer verifier needs the following mathematical operations. Validate the DAG and its generator alphabet. Evaluate each live node by (16). Form every comparison using the inverse

rule. Reject an odd s_j or a central -1 . Compute d by integer gcd. When $d > 0$, obtain e from any odd normalized exponent and test (18). Finally test the open-interval count (19) or verify the supplied rational phase directly. For $d = 0$, check that every comparison is the identity normal form.

The topological caller must separately verify the original diagram and the presentation/meridian trace. It is useful to keep this caller outside the small algebraic verifier: an arithmetic certificate cannot establish an arbitrary assertion about a knot diagram it has never read. Similarly, a digest binds data to a certificate but does not prove that a transformation of those data preserved the knot group.

References

- [1] V. Reshetnikov, ProveIt repository, *Continuing group search across word representations*, Topology/UnknotRecognition/synthesis/adaptive_group.tex, inspected at commit f93328fbf5b576056ad3d8110c2102a46529e514, 8 October 2026.
<https://github.com/VladimirReshetnikov/ProveIt>
- [2] V. Reshetnikov, ProveIt repository, Topology/UnknotRecognition/fast/fastunknot/compressed_words.py, inspected at commit 8170a64c7e72b890972acf200812d6dc77ae436f, 8 October 2026. Same repository as above.
- [3] S. M. Meesum and T. V. H. Prathamesh, *Unknot Recognition Through Quantifier Elimination*, arXiv:1803.00413v1, 2018. In particular, the representation reduction and existential-real complexity input.
<https://arxiv.org/abs/1803.00413>
- [4] J. Renegar, *On the computational complexity and geometry of the first-order theory of the reals. Part I: Introduction. Preliminaries. The geometry of semi-algebraic sets. The decision problem for the existential theory of the reals*, Journal of Symbolic Computation **13** (1992), 255–299. The decision bound is also stated in Proposition 7 of [3].
- [5] P. B. Kronheimer and T. S. Mrowka, *Witten’s conjecture and Property P*, Geometry & Topology **8** (2004), 295–310.
<https://arxiv.org/abs/math/0311489>
- [6] Y. Xie and B. Zhang, *On meridian-traceless $SU(2)$ -representations of link groups*, arXiv:2104.04839v2, 2021. We use the specialization to knots.
<https://arxiv.org/abs/2104.04839>
- [7] R. Zentner, *A class of knots with simple $SU(2)$ representations*, Selecta Mathematica (N.S.) **23** (2017), 2219–2242; arXiv:1501.02504.
<https://arxiv.org/abs/1501.02504>
- [8] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, July 2026.
<https://arxiv.org/abs/2607.23350>
- [9] C. Musick, *Locally Minimal Bridge Presentations of Knots*, arXiv:2609.06492v2, September 2026. The claimed polynomial recognition theorem is noted, not assumed or verified in this article.
<https://arxiv.org/abs/2609.06492v2>